

Priority Queues and Heaps

Jianguo Lu

February 12, 2025

Overview

Priority Queue

Heaps

Priority queue ADT

`insert( $k$ ,  $v$ )` : Creates an entry with key k and value v in the priority queue.

`min()` : Returns an entry (k,v) having minimal key;

`removeMin()` : Removes and returns an entry (k,v) having minimal key;

`size()` : Returns the number of entries in the priority queue.

`isEmpty()` : Check whether the priority queue is empty.

Implementations of Priority Queue Using Linked List

- ▶ Sorted
- ▶ Unsorted

	min() and removeMin()	insert(k,v)
sorted	$O(1)$	$O(n)$
unsorted	$O(n)$	$O(1)$

- ▶ How to solve the dilemma?
- ▶ We only need the data partially sorted
- ▶ As long as the min is at the top

Heap

A (binary) heap is a binary tree that satisfies the following two properties:

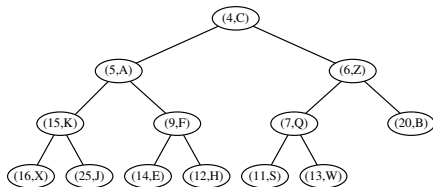
Heap-Order Property

- ▶ Parent is smaller than its children (for MinHeap)
 - ▶ This is to guarantee that the root is the min

Complete Binary Tree Property

A heap T with height h is a complete binary tree:

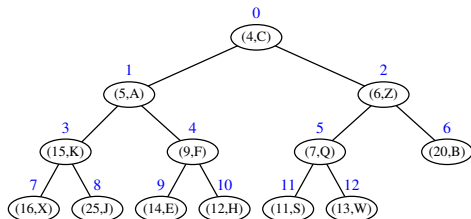
- ▶ levels $0, 1, 2, \dots, h - 1$ of T have the maximal number of nodes possible
- ▶ Remaining nodes at level h reside in the leftmost possible positions.
- ▶ This is to guarantee a space efficient array representation.



Array based representation

Array starting from index 0

- ▶ $a[0]$ is the root
- ▶ $a[(k+1)/2-1]$ is the parent of $a[k]$
- ▶ $a[2k+1]$ is the left child, $a[2k+2]$ is the right child
- ▶ MinHeap example from the book:



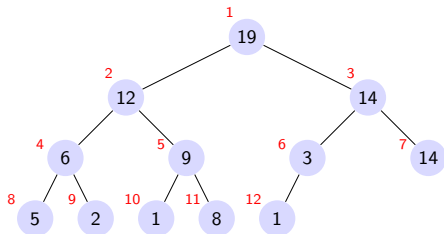
- ▶ It's array implementation:

(4,C)	(5,A)	(6,Z)	(15,K)	(9,F)	(7,Q)	(20,B)	(16,X)	(25,J)	(14,E)	(12,H)	(11,S)	(13,W)
0	1	2	3	4	5	6	7	8	9	10	11	12

Array based representation in our code

Array starting from index 1

- ▶ $a[1]$ is the root
- ▶ $a[k/2]$ is the parent of $a[k]$
- ▶ $a[2k]$ is the left child, $a[2k+1]$ is the right child
- ▶ A MaxHeap example:



- ▶ It's array representation:

i	1	2	3	4	5	6	7	8	9	10	11	12
$A[i]$	19	12	14	6	9	3	14	5	2	1	8	1

Algorithms, 4th Edition by Robert Sedgewick and Kevin Wayne.

<https://algs4.cs.princeton.edu/24pq/>

Height of a heap

Proposition 9.2

A heap T storing n entries has height $h = \lfloor \log n \rfloor$.

- Justification: From the fact that T is complete, we know that the number of nodes in levels 0 through $h - 1$ of T is

$$2^0 + 2^1 + 2^2 + \cdots + 2^{h-1} = 2^h - 1 \quad (1)$$

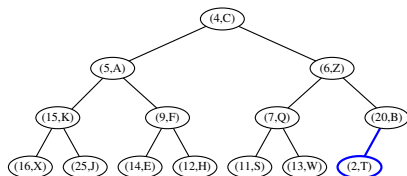
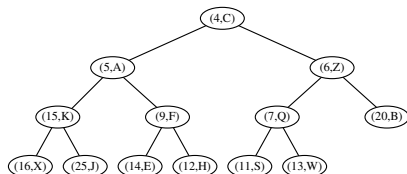
The number of nodes in level h is at least 1 and at most 2^h . Therefore

$$n \geq 2^h - 1 + 1 = 2^h \quad (2)$$

$$n \leq 2^h - 1 + 2^h = 2^{h+1} - 1. \quad (3)$$

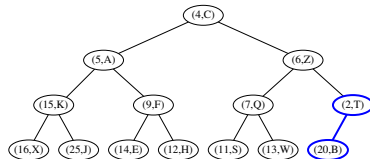
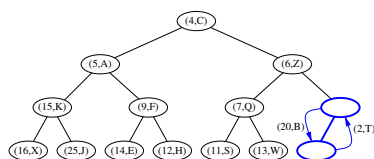
$$\log(n + 1) - 1 \leq h \leq \log n \quad (4)$$

Insertion (in minHeap)



- ▶ Add the element at the very end
- ▶ Upward bubbling

Up-heap bubbling after insertion



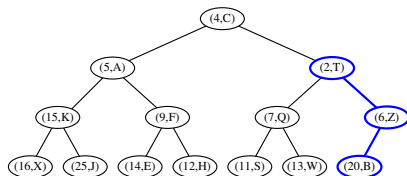
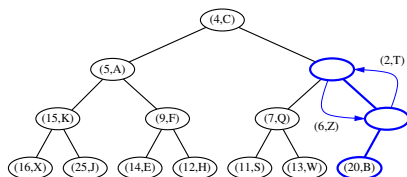
```
► swimPseudocode(int k)
    while k has parents and smaller than parent
        swap(k, parent)
        continue from parent
```

► Remember that parent index is $k/2$:

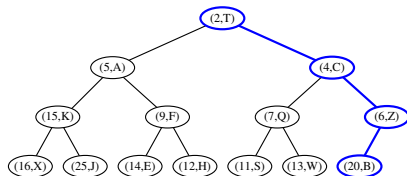
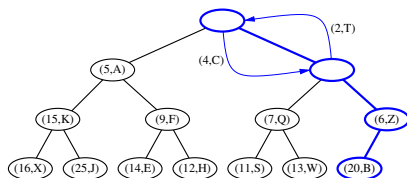
```
► void swim(int k) {
    while (k > 1 && less(k, k/2)) {
        exch(k, k/2);
        k = k/2;
    }
}
```

Note that this is for minHeap. MaxHeap should reverse the comparison.

Up-heap bubbling after insertion

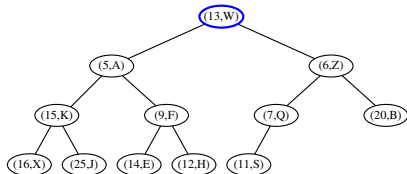
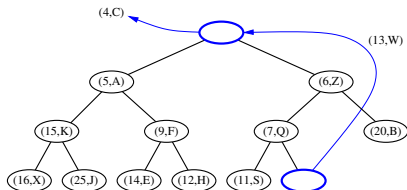


Up-heap bubbling after insertion



RemoveMax or RemoveMin

- ▶ Remove the top element
- ▶ Replace it with the last element
- ▶ Downward bubbling



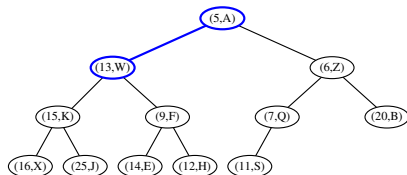
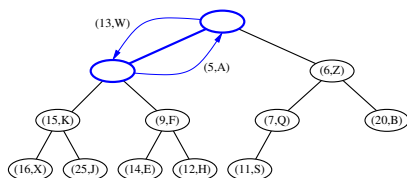
Downward bubbling for minHeap

Downward bubbling (for minHeap)

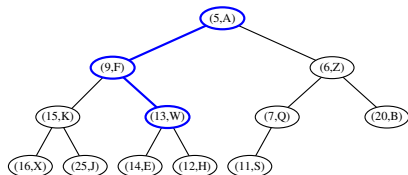
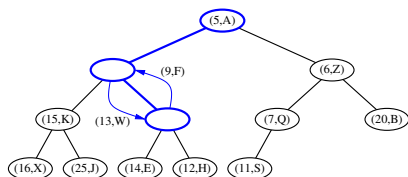
```
Pseudocode_sink(int k)
    while k has child
        Let j be the index of the smaller child
        if a[k] larger than a[j]
            exch(k, j);
        continue with j

private void sink(int k) {
    while (2*k <= n) { //n is the Heap size
        int j = 2*k;
        if (j < N && less(j+1, j)) j++;
        if (!less(j, k)) break;
        exch(k, j);
        k = j;
    }
}
```

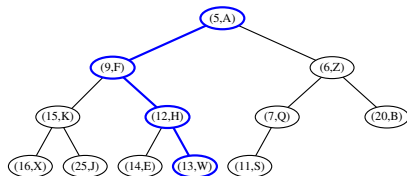
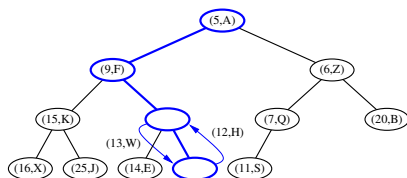
Downward bubbling



Downward bubbling in minHeap



Downward bubbling



Analysis of Heap operations

Method Running Time	
size, isEmpty	$O(1)$
max/min	$O(1)$
insert	$O(\log n)$
removeMax/removeMin	$O(\log n)$

Top-down heap construction (Bad one)

Build a max-heap using the insertion approach

(view the animation using Acrobat Reader)

Bottom-Up Heap Construction: a better approach

```
void constructHeap(int[] a)
    int n = a.length;
    for (int k = n/2; k >= 1; k--)
        sink(k);
```

Time complexity of Heap construction

layer	#nodes	# comparisons/exchanges
4	2^0	$4 \times 2^{h-4}$
3	2^1	$3 \times 2^{h-3}$
4	2^2	$2 \times 2^{h-2}$
1	2^3	$1 \times 2^{h-1}$
i	2^{h-i}	$i \times 2^{h-i}$

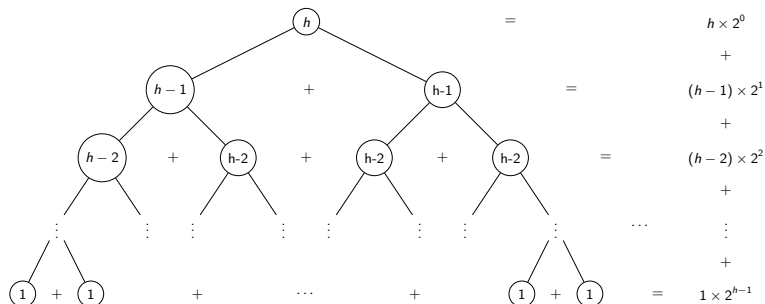
$$T = \sum_{i=0}^h i \times 2^{h-i} = 2^h \sum_{i=0}^h \frac{i}{2^i} \approx n \sum_{i=0}^h \frac{i}{2^i} \quad (2^h \approx n)$$
$$\approx n \times 2 \quad (\text{arithmetico-geometric series})$$

Arithmetic-geometric sequence

$$\sum_{i=0}^k \frac{i}{2^i} \approx 2 \quad (5)$$

Time complexity of Heap construction

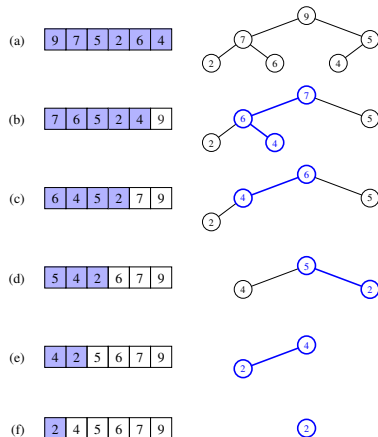
- ▶ Suppose it is a *perfect* binary tree.
- ▶ bottom layer is not drawn.



Heap sort

- ▶ Continuously select the top element
- ▶ Note that it is an *in place* sorting
 - ▶ in-place sort: occupy the same storage as the original ones.

```
static void heapSort(int[] a)
{
    constructHeap(a);
    int n = a.length;
    while (n > 1) {
        exch(a, 1, n);
        n--;
        sink(a, 1, n);
    }
}
```



Take-aways

- ▶ What are the major operations in Priority Queue ADT? (`max()`, `removeMax()`)
- ▶ How do we implement it? (Heap)
- ▶ Why do we need the Heap data structure?
- ▶ How to implement a Heap using an Array? Why Array? (Complete binary tree).
- ▶ What is the time complexity for `removeMax` and `Max` operations in a heap?
- ▶ What is the cost to construct heap? (bad one and good one)
- ▶ Can you write the `heapSort` algorithm? What is the time complexity for `heapSort`?
- ▶ Readings:
 - ▶ Heap sort in book *Algorithms, 4th Edition* by Robert Sedgewick and Kevin Wayne. <https://algs4.cs.princeton.edu/24pq/>
 - ▶ Goodrich et al. PP 360-389